

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Threading.Tasks;
5 using Npgsql;
6 namespace FormApp
7 {
8     2 references
9     public class EmployeeService
10    {
11        5 references
12        private string connectionString = "Server=localhost;port=5432;Database=stud2025;User Id=postgres;Password=Admin@123;";
13
14        // Create
15        1 reference
16        public void AddEmployee(Employee emp)
17        {
18            var conn = new NpgsqlConnection(connectionString);
19            try
20            {
21                conn.Open();
22                using var cmd = new NpgsqlCommand("INSERT INTO t_employee(empid, empname, salary, deptno) VALUES (@id, @name, @salary, @deptno)", conn);
23                cmd.Parameters.AddWithValue("id", emp.EmpId);
24                cmd.Parameters.AddWithValue("name", emp.EmpName);
25                cmd.Parameters.AddWithValue("salary", emp.Salary);
26                cmd.Parameters.AddWithValue("deptno", emp.DeptNo);
27                cmd.ExecuteNonQuery();
28            }
29            catch (Exception ex)
30            {
31                MessageBox.Show("Error adding employee: " + ex.Message);
32            }
33            finally
34            {
35                conn.Close();// Any cleanup code if necessary
36            }
37        }
38        // Read all
39        1 reference
40        public List<Employee> GetAllEmployees()
41        {
42            var list = new List<Employee>();
43            var conn = new NpgsqlConnection(connectionString);
44            conn.Open();
45            using var cmd = new NpgsqlCommand("SELECT empid, empname, salary, deptno FROM t_employee", conn);
46            using var reader = cmd.ExecuteReader();
47            while (reader.Read())
48            {

```

```

44            while (reader.Read())
45            {
46                list.Add(new Employee
47                {
48                    EmpId = reader.GetInt32(0),
49                    EmpName = reader.GetString(1),
50                    Salary = reader.GetDecimal(2),
51                    DeptNo = reader.IsDBNull(3) ? 0 : reader.GetInt32(3)
52                });
53            }
54            return list;
55        }
56    }
57
58    // Read single
59    0 references
60    public Employee GetEmployee(int empId)
61    {
62        try
63        {
64            using var conn = new NpgsqlConnection(connectionString);
65            conn.Open();
66            using var cmd = new NpgsqlCommand("SELECT empid, empname, salary, deptno FROM t_employee WHERE empid = @id", conn);
67            cmd.Parameters.AddWithValue("id", empId);
68            using var reader = cmd.ExecuteReader();
69            if (reader.Read())
70            {
71                return new Employee
72                {
73                    EmpId = reader.GetInt32(0),
74                    EmpName = reader.GetString(1),
75                    Salary = reader.GetDecimal(2),
76                    DeptNo = reader.IsDBNull(3) ? 0 : reader.GetInt32(3)
77                };
78            }
79        }
80        catch (Exception ex)
81        {
82            MessageBox.Show("Error retrieving employee: " + ex.Message);
83        }
84        return null;
85    }
86
87    // Update
88    1 reference
89    public bool UpdateEmployee(Employee emp)
90    {
91        try
92        {
93            using var conn = new NpgsqlConnection(connectionString);
94            conn.Open();
95            using var cmd = new NpgsqlCommand("UPDATE t_employee SET empname=@name, salary=@salary, deptno=@deptno WHERE empid=@id", conn);
96            cmd.Parameters.AddWithValue("id", emp.EmpId);
97            cmd.Parameters.AddWithValue("name", emp.EmpName);
98            cmd.Parameters.AddWithValue("salary", emp.Salary);
99            cmd.Parameters.AddWithValue("deptno", emp.DeptNo);
100            int affected = cmd.ExecuteNonQuery();
101            return affected > 0;
102        }
103        catch (Exception ex)
104        {
105            MessageBox.Show("Error updating employee: " + ex.Message);
106            return false;
107        }
108    }
109
110    // Delete
111    1 reference
112    public bool DeleteEmployee(int empId)
113    {
114        try
115        {
116            using var conn = new NpgsqlConnection(connectionString);
117            conn.Open();
118            using var cmd = new NpgsqlCommand("DELETE FROM t_employee WHERE empid = @id", conn);
119            cmd.Parameters.AddWithValue("id", empId);
120            int affected = cmd.ExecuteNonQuery();
121            return affected > 0;
122        }
123        catch (Exception ex)
124        {
125            MessageBox.Show("Error deleting employee: " + ex.Message);
126            return false;
127        }
128    }
129
130    }
131
132    }
133
134    }
135
136    }
137
138    }
139
140    }
141
142    }
143
144    }
145
146    }
147
148    }
149
150    }
151
152    }
153
154    }
155
156    }
157
158    }
159
160    }
161
162    }
163
164    }
165
166    }
167
168    }
169
170    }
171
172    }
173
174    }
175
176    }
177
178    }
179
180    }
181
182    }
183
184    }
185
186    }
187
188    }
189
190    }
191
192    }
193
194    }
195
196    }
197
198    }
199
200    }
201
202    }
203
204    }
205
206    }
207
208    }
209
210    }
211
212    }
213
214    }
215
216    }
217
218    }
219
220    }
221
222    }
223
224    }
225
226    }
227
228    }
229
230    }
231
232    }
233
234    }
235
236    }
237
238    }
239
240    }
241
242    }
243
244    }
245
246    }
247
248    }
249
250    }
251
252    }
253
254    }
255
256    }
257
258    }
259
260    }
261
262    }
263
264    }
265
266    }
267
268    }
269
270    }
271
272    }
273
274    }
275
276    }
277
278    }
279
280    }
281
282    }
283
284    }
285
286    }
287
288    }
289
290    }
291
292    }
293
294    }
295
296    }
297
298    }
299
300    }
301
302    }
303
304    }
305
306    }
307
308    }
309
310    }
311
312    }
313
314    }
315
316    }
317
318    }
319
320    }
321
322    }
323
324    }
325
326    }
327
328    }
329
330    }
331
332    }
333
334    }
335
336    }
337
338    }
339
340    }
341
342    }
343
344    }
345
346    }
347
348    }
349
350    }
351
352    }
353
354    }
355
356    }
357
358    }
359
360    }
361
362    }
363
364    }
365
366    }
367
368    }
369
370    }
371
372    }
373
374    }
375
376    }
377
378    }
379
380    }
381
382    }
383
384    }
385
386    }
387
388    }
389
390    }
391
392    }
393
394    }
395
396    }
397
398    }
399
400    }
401
402    }
403
404    }
405
406    }
407
408    }
409
410    }
411
412    }
413
414    }
415
416    }
417
418    }
419
420    }
421
422    }
423
424    }
425
426    }
427
428    }
429
430    }
431
432    }
433
434    }
435
436    }
437
438    }
439
440    }
441
442    }
443
444    }
445
446    }
447
448    }
449
450    }
451
452    }
453
454    }
455
456    }
457
458    }
459
460    }
461
462    }
463
464    }
465
466    }
467
468    }
469
470    }
471
472    }
473
474    }
475
476    }
477
478    }
479
480    }
481
482    }
483
484    }
485
486    }
487
488    }
489
490    }
491
492    }
493
494    }
495
496    }
497
498    }
499
500    }
501
502    }
503
504    }
505
506    }
507
508    }
509
510    }
511
512    }
513
514    }
515
516    }
517
518    }
519
520    }
521
522    }
523
524    }
525
526    }
527
528    }
529
530    }
531
532    }
533
534    }
535
536    }
537
538    }
539
540    }
541
542    }
543
544    }
545
546    }
547
548    }
549
550    }
551
552    }
553
554    }
555
556    }
557
558    }
559
560    }
561
562    }
563
564    }
565
566    }
567
568    }
569
570    }
571
572    }
573
574    }
575
576    }
577
578    }
579
580    }
581
582    }
583
584    }
585
586    }
587
588    }
589
590    }
591
592    }
593
594    }
595
596    }
597
598    }
599
600    }
601
602    }
603
604    }
605
606    }
607
608    }
609
610    }
611
612    }
613
614    }
615
616    }
617
618    }
619
620    }
621
622    }
623
624    }
625
626    }
627
628    }
629
630    }
631
632    }
633
634    }
635
636    }
637
638    }
639
640    }
641
642    }
643
644    }
645
646    }
647
648    }
649
650    }
651
652    }
653
654    }
655
656    }
657
658    }
659
660    }
661
662    }
663
664    }
665
666    }
667
668    }
669
670    }
671
672    }
673
674    }
675
676    }
677
678    }
679
680    }
681
682    }
683
684    }
685
686    }
687
688    }
689
690    }
691
692    }
693
694    }
695
696    }
697
698    }
699
700    }
701
702    }
703
704    }
705
706    }
707
708    }
709
710    }
711
712    }
713
714    }
715
716    }
717
718    }
719
720    }
721
722    }
723
724    }
725
726    }
727
728    }
729
730    }
731
732    }
733
734    }
735
736    }
737
738    }
739
740    }
741
742    }
743
744    }
745
746    }
747
748    }
749
750    }
751
752    }
753
754    }
755
756    }
757
758    }
759
760    }
761
762    }
763
764    }
765
766    }
767
768    }
769
770    }
771
772    }
773
774    }
775
776    }
777
778    }
779
780    }
781
782    }
783
784    }
785
786    }
787
788    }
789
790    }
791
792    }
793
794    }
795
796    }
797
798    }
799
800    }
801
802    }
803
804    }
805
806    }
807
808    }
809
810    }
811
812    }
813
814    }
815
816    }
817
818    }
819
820    }
821
822    }
823
824    }
825
826    }
827
828    }
829
830    }
831
832    }
833
834    }
835
836    }
837
838    }
839
840    }
841
842    }
843
844    }
845
846    }
847
848    }
849
850    }
851
852    }
853
854    }
855
856    }
857
858    }
859
860    }
861
862    }
863
864    }
865
866    }
867
868    }
869
870    }
871
872    }
873
874    }
875
876    }
877
878    }
879
880    }
881
882    }
883
884    }
885
886    }
887
888    }
889
890    }
891
892    }
893
894    }
895
896    }
897
898    }
899
900    }
901
902    }
903
904    }
905
906    }
907
908    }
909
910    }
911
912    }
913
914    }
915
916    }
917
918    }
919
920    }
921
922    }
923
924    }
925
926    }
927
928    }
929
930    }
931
932    }
933
934    }
935
936    }
937
938    }
939
940    }
941
942    }
943
944    }
945
946    }
947
948    }
949
950    }
951
952    }
953
954    }
955
956    }
957
958    }
959
960    }
961
962    }
963
964    }
965
966    }
967
968    }
969
970    }
971
972    }
973
974    }
975
976    }
977
978    }
979
980    }
981
982    }
983
984    }
985
986    }
987
988    }
989
990    }
991
992    }
993
994    }
995
996    }
997
998    }
999
1000   }

```